Date. No.

5-dars _ 5-dars _ 5-dars

Logistic Regression vs Linear Regression

1) O'xshash tomonlari:

1) Supervised Learning → Ikkalasi ham o'qitilgan (label bor) dataset bilan ishlaydi.

2) Chiziqli model (linear model) → Ikkalasi ham input featurelardan chiziqli kombinatsiya qiladi:

Formula: $y = w_1x_1 + w_2x_2 + \dots + b$

3) Ko'p feature bilan ishlay oladi → Bir nechta input ($x_1, x_2, x_3 \dots$) qabul qiladi

4) Training jarayoni o'xshash → Gradient Descent yoki boshqa optimizatsiya usullaridan foydalanadi.

2) Farqli tomonlari:

1) Maqsadi → Linear Regression (son ~ continuous value) bashorat qiladi (Masalan: uy narxi, temperatura)

Logistic Regression (sinf ~ class) aniqlaydi (Masalan: spam/not spam)

2) Natija (Output) → Linear Regression (Natija istalgan real son bo'ladi) (Masalan: 120.8, 300.2)

Logistic Regression → Natija 0 va 1 orasida ehtimollik bo'ladi (masalan: 0.87 → 87%)

3) Funksiya:

- Linear Regression (oddiy chiziqli funksiya) $y = wx + b$
- Logistic Regression (Sigmoid funksiya ishlatadi) $\sigma(x) = \frac{1}{1+e^{-x}}$

4) Loss Function:

- Linear Regression → MSE (Mean Squared Error)
- Logistic Regression → Log Loss (Cross-Entropy)

Algorithm → Istalgan mantiqqa suyangan algoritm

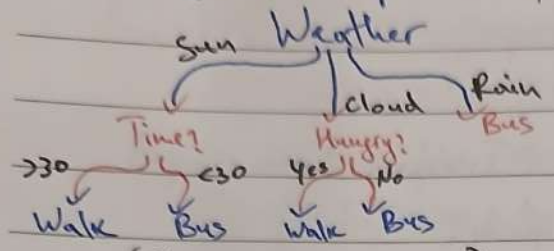
Model → Qolganlar ham foydalana oladigan algoritm

Date.

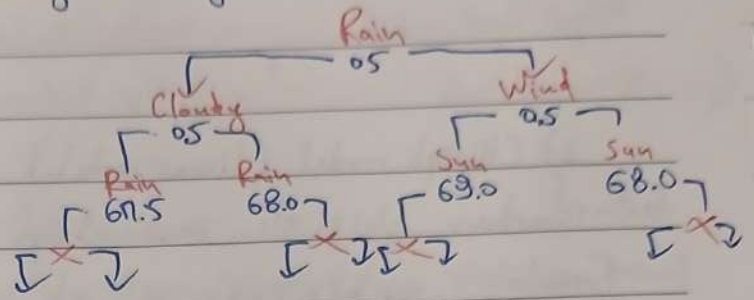
No.

Tree-Based Algorithms

• Conditionga kora datasetni kichik qismlarga agratadi va shunga kora predict qiladigani Algorithm oilalaridan biri



(Classification uchun)



(Regression uchun)

#1) Eng muhim featureni tanlab oladi

2) Ikkinchi bolib 2-chi darajada muhim featureni oladi

(Parametr → max_depth=3)

Orta arifmetic orqali Regression da hambir xil lekin Javob olishdagina farqi bor:

Classification → Eng kop qaytarilganni javob qilib oladi:

Regression → Hammasini qoshib orta arifmetikni qiymat oladi.

• Tree Based algorithm (Classification + Regression)

machinelearning Ensamble algorithm da hambor

- Decision Tree
- Random Forest (DTree1, DTree2, DTree3)
- Gradient Boosting Machine
- Ada Boost

≈ Linear Modellardan farqi

• Non-Linear oilasi / • Non-linear xolatlarda yuqori natija beradi

• Scaling kerak emas (Qiymatga qarabemas turgan yeriga qarab olshaydi)

Date. No.

Tree Based modellar analiz

```
import pandas as pd
df = pd.read_csv('Fayl.csv')
df.info() → df.head()
# Data b/n tanishuv
df.drop('column_name', axis=1, inplace=True) → df.head()
# Data Preprocessing
df.isnull().sum()
#Missing + Function
```

#Missing + Function

```
def toldir_missing(df):
    for col in df.columns:
        if df[col].isnull().any():
            if df[col].dtype=="object":
                df[col] = df[col].fillna(df[col].mode()[0])
            else:
                df[col] = df[col].fillna(df[col].mean())
return df
df = toldir_missing(df)
```

Date.

No.

```
# Encoding + Function                                  # Encoding + Function
from sklearn.preprocessing import LabelEncoder
def encodla(df):
    encoder = LabelEncoder()
    for col in df.columns:
        if df[col].dtype == 'object':
            if df[col].nunique() < 5:
                dummies = pd.get_dummies(df[col], prefix=col, dtype=int)
                df = pd.concat([df.drop(columns=[col]), dummies], axis=1)
            else:
                df[col] = encoder.fit_transform(df[col])
    return df
df = encodla(df)
df.head() -> df.head(df.info())

# Scaling + Function                                   # Scaling + Function
from sklearn.preprocessing import MinMaxScaler, StandardScaler
def scaling_qil(df):
    scaler = MinMaxScaler()
    num_col = df.select_dtypes(include=['int32','int64']).columns.drop('Target')
    df[num_col] = scaler.fit_transform(df[num_col])
    return df
df = scaling_qil(df)
# Biz Tree-Basedda Scaling qilmasak ham bo'ladi!!!
```

Date. No.

```
# Training
x= df.drop('Target', axis=1)
y= df['Target']
    from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=42)
df['Target'].value_counts()
df.info() → df.head()
#
from sklearn.tree import DecisionTreeRegression (DecisionTreeClassifier)
dt_model = DecisionTreeRegression()
dt_model
# Fit --- yani training qilish
dt_model.fit(x_train, y_train)
# Predict
y_pred = dt_model.predict(x_test)
y_pred
# Model evaluation
from sklearn.metrics import r2_score, mean_absolute_error,
                            mean_squared_error,
                            root_mean_squared_error

r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = root_mean_squared_error(y_test, y_pred)
print('r2:', r2)
print('mae:', mae)
print('mse:', mse)
print('rmse:', rmse)
```

Date.

No.

```
# Shuni compare LinearRegression bilan!
from sklearn.linear_model import LinearRegression
    lr_model = LinearRegression()
  lr_model.fit(x_train, y_train)
    y_pred = lr_model.predict (x_test)
        y_pred
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = root_mean_squared_error(y_test, y_pred)
print('r2: ', r2)
print('mae: ', mae)
print('mse: ', mse)
print('rmse: ', rmse)
        y.nunique()    # classlar soni
```

Date. No.

```
# Decision Tree Classifier
from sklearn.tree import DecisionTreeClassifier
dt_cl = DecisionTreeClassifier()
dt_cl.fit(x_train, y_train)        # training
y_pred = dt_cl.predict(x_test)     # prediction
from sklearn.metrics import classification_report, confusion_matrix
class_report = classification_report(y_test, y_pred)
print(class_report)
y.value_counts()
```

```
# Logistic Regression
from sklearn.linear_model import LogisticRegression
lg_reg = LogisticRegression()
lg_reg.fit(x_train, y_train)
y_pred = lg_reg.predict(x_test)
class_report = classification_report(y_test, y_pred)
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.metrics import accuracy_score
acc_score = accuracy_score(y_test, y_pred)
print(acc_score)
```

⇒ Linear tomonga yaxshi natija chiqdi (Linear = Complex)